Data Structures II

CPSC 482

Dr. Hossain

DSM Project Report

Name	**Email**	**Student ID**
Simon Kraft	kraft@unbc.ca	230171256
Joshua Payne	paynej0@unbc.ca	230152032
El Sall	esall@email.com	230158722

March 30, 2026

Abstract

The Design Structure Matrix (DSM) is a compact representation of task dependencies in large-scale engineering and software systems. Efficient reordering of DSM tasks to minimize feed-back dependencies is a computationally challenging problem with significant practical importance. In this report, we present an implementation of two classical strongly connected component (SCC) algorithms — Tarjan's algorithm and the Sharir-Kosaraju algorithm — using the Compressed Sparse Column (CSC) data structure in order to decompose DSMs into block lower triangular form. We evaluate both algorithms on a suite of benchmark sparse matrices and compare their performance in terms of running time, number of SCCs identified, and solution quality as measured by the number of feed-back marks and total feed-back distance. Our results demonstrate that [PLACEHOLDER: insert key finding, e.g., "both algorithms produce identical partitionings, with Tarjan's algorithm achieving a modest runtime advantage on larger inputs"].

Contents

1 Introduction

Modern software systems and engineering projects are constantly growing in complexity, often comprising hundreds or even thousands of interdependent tasks that require the collaboration of many participants from diverse backgrounds. Traditional management tools such as Gantt charts and Critical Path Method (CPM) networks are well-suited for modeling sequential and parallel task executions. However, they fail for interdependent tasks, i.e., when the output of one task is needed as input for another task, and vice versa. As these tasks are especially common in complex product development (PD), project managers require tools capable of modeling interdepdencies and identifying a task ordering that minimizes potential conflicts in information flow [8].

An effective tool for representing the pairwise information flow between a set of tasks is a square matrix called the *design structure matrix* or *dependency structure matrix* (DSM) [3]. If a project is composed of n tasks, a DSM models the information flow between these tasks in a binary $n \times n$ matrix, where the rows and columns are both labeled by the same ordered list of tasks. Marks in the DSM indicate the relationship between two tasks. The marks in a single row of the DSM show all the tasks that provide input that is needed to perform the task in the corresponding row. Similarly, marks in a single column display all the tasks that receive information from the corresponding task in that column [8]. For instance, the (i, j)-th entry of the DSM is 1, if task i depends on task j, and it is 0 if task i can be completed independent of task j. Furthermore, the diagonal entries do not represent dependencies with other tasks and are therefore often disregarded. Equivalent to its matrix representation, a DSM can also be displayed as a directed graph [1].

If the ordering of tasks along the axes is interpreted as the sequence in which the tasks are executed, then three different types of dependencies can be identified for a pair of tasks [8].

- **Feed-Forward**: Marks that represent the forward transfer of information. Task j supplies information for task i and is scheduled before it, i.e., $j < i$. The pair (i, j) is located below the diagonal. These dependencies are easy to solve as task i just waits for task j to finish.

- **Feed-Back**: Marks that represent the backward transfer of information. Task j supplies information for task i but is scheduled afterwards, i.e., $i < j$. The pair (i, j) is located above the diagonal. This means that the required inputs (task j) are not available at the time of executing task i, making it necessary to make assumptions about the inputs that may not hold if task j is finally executed. In the worst case, the dependent task i needs to be re-executed.

- **Coupled**: Task i and j are mutually dependent and both require input from each other, i.e., the pairs (i, j) and (j, i) both exist in the DSM. These tasks require iteration as neither task can be started first without making assumptions about the output of the other task.

An important objective from a project manager's perspective is reorder the tasks to reduce the number of feed-back marks to minimize the potential conflicts that could arise during the execution stage. This process is regarded as *partitioning* [5, 9].

In the ideal scenario, the permutation of the rows and columns leads to a lower triangular form in the DSM, i.e., all feed-back marks get eliminated. However, not all DSMs can be permuted to lower triangular form [2, 8]. Especially for complex engineering systems it is highly unlikely that all feed-back marks can be moved underneath the diagonal. If the underlying directed graph describing the DSM contains at least one cycle, the best achievable result is the *block lower triangular form* (BLTF), in which all vertices participating in cycles are gathered into diagonal blocks [2]. Furthermore, the second objective is to minimize the total distance of the remaining feed-back marks from the diagonal, as smaller and more tightly packed diagonal blocks cause fewer tasks to be involved in each iteration cycle, leading to a faster development process [1].

With these two objectives at hand, the DSM partitioning problem can be stated as follows. Let $A \in \{0,1\}^{n\times n}$ be a binary matrix representing the DSM and let $P \in \{0,1\}^{n\times n}$ be a permutation matrix obtained by permuting the columns/ rows of the identity matrix I_n. The symmetrically permuted matrix $A' = P^\top AP$ represents the reordered DSM, where the goal is to find the permutation matrix P that minimizes two quantities [2]:

1. The number of feed-back marks |FBM|, where

$$\text{FBM} = \{a'_{ij} \neq 0 \mid j > i,\ i, j = 1, 2, \ldots, n\}$$

2. The total feed-back distance TFBD of feed-back marks from the diagonal

$$\text{TFBD} = \sum_{a'_{ij} \in \text{FBM}} (j - i),$$

Over the years, several algorithmic approaches have been proposed in the literature to solve the DSM partitioning problem [1, 8]. However, most methods share the same structure and only differ in the way they identify *information cycles*, i.e. cycles of two or more mutually dependent tasks. Usually, the partitioning starts by moving tasks with no incoming dependencies (empty rows) to the top of the DSM, and moving tasks with no outgoing dependencies to the bottom of the DSM. Once the these tasks are rearranged, they are removed from the DSM with all their corresponding marks. These steps are repeated until no further reordering is possible [8]. Any remaining task in the DSM must then be part of at least one information cycle, which must be identified explicitly. Three classical methods have been proposed for this [1, 9]:

- **Path Searching**: Every remaining activity has an off-diagonal mark [1]. An activity is arbitrarily chosen and a list of successors is generated by tracing the information flow until a task in encountered twice [4, 5]. All tasks between the first and second occurrence constitute an information cycle and are grouped together [8]. The approach can also be applied recursively to reveal sub-cycles [1].
- **Powers of the Adjacency Matrix**: The binary DSM is raised to the n-th power. A non-zero diagonal entry indicates that the corresponding task can reach itself in n steps, which confirms a cycle [7, 8]. However, finding blocks of coupled activities through this method is computationally expensive, especially for large matrices [1].

- **Depth-First Search**: The most efficient way to identify subsets of coupled activities is Tarjan's depth-first search algorithm [1, 6]. Following the outputs of each task detects dependency paths that cycle back to the same task.

When all information cycles are identified, the involved tasks are grouped together to be treated as a single representative block [8]. Then the procedure restarts on the remaining tasks, which ultimately yields the block lower triangular form of the DSM.

The remainder of this report is structured as follows: Section 2 describes CSC matrix data structure and the algorithms that were used to permute the DSM, including their asymptotic runtime and space complexities. Section 3 presents the results of our numerical experiments on benchmark matrices in terms of solution quality and running time. Section 4 summarizes our findings and provides future research directions of this work.

2 Description of Solution

3 Numerical Results

4 Conclusion

References

[1] S. Eppinger and T. Browning. *Design Structure Matrix Methods and Applications*. May 2012. ISBN: 9780262301428 (cit. on pp. 3–5).

[2] S. Hossain. "Hints on Data Structure and Algorithms for Term Project". Course document, CPSC 482/682, University of Northern British Columbia. 2026 (cit. on p. 4).

[3] S. Hossain. "Term Project for CPSC482/682 Data Structures II". Course document, CPSC 482/682, University of Northern British Columbia. 2026 (cit. on p. 3).

[4] R. Sargent and A. Westerberg. "SPEED-UP in Chemical Engineering Design". In: *Transactions of the Institution of Chemical Engineers* 42 (Jan. 1964), p. 190 (cit. on p. 4).

[5] D. V. Steward. "The design structure system: A method for managing the design of complex systems". In: *IEEE Transactions on Engineering Management* EM-28.3 (1981), pp. 71–74 (cit. on pp. 3, 4).

[6] R. Tarjan. "Depth-First Search and Linear Graph Algorithms". In: *SIAM Journal on Computing* 1.2 (1972), pp. 146–160. eprint: https://doi.org/10.1137/0201010. URL: https://doi.org/10.1137/0201010 (cit. on p. 5).

[7] J. N. Warfield. "Binary Matrices in System Modeling". In: *IEEE Transactions on Systems, Man, and Cybernetics* SMC-3.5 (1973), pp. 441–449 (cit. on p. 4).

[8] A. Yassine. "An Introduction to Modeling and Analyzing Complex Product Development Processes Using the Design Structure Matrix (DSM) Method". In: 2001. URL: https://api.semanticscholar.org/CorpusID:13887392 (cit. on pp. 3–5).

[9] A. Yassine, D. Falkenburg, and K. Chelst. "Engineering design management: An information structure approach". In: *International Journal of Production Research* 37.13 (1999), pp. 2957–2975. eprint: https://doi.org/10.1080/002075499190374. URL: https://doi.org/10.1080/002075499190374 (cit. on pp. 3, 4).